Assignment 3 **Implementation of HTTP Client using TCP Sockets**

Deadline: 20/7/26 (First Batch) 22/7/26 (Second Batch)

Mandatory Program

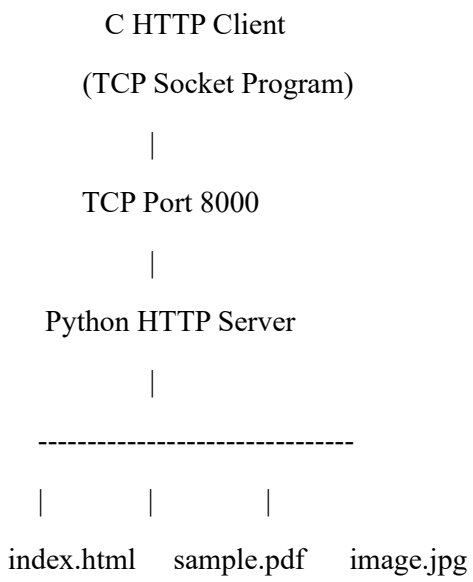
a) Develop a TCP socket-based HTTP client application in C that accepts a URL from the user, establishes a connection with the corresponding web server, sends an HTTP GET request, retrieves the requested webpage or file, and stores the downloaded content locally.

b) Develop an HTTP client using TCP sockets in C to communicate with a Python-based web server and download web resources (HTML, text, images, and PDF files).

In both cases, the client should analyse the performance of the communication by measuring response time, download time, data size, and throughput.

For **Python-based web server** , you need to **deploy (host) some files** in the server directory. The C TCP client will request those files using the URL, and the Python server will send them back.

Overall Structure



Step 1: Create Web Server Folder

Create a folder:

mkdir CN_Web_Server

cd CN_Web_Server

Place files inside:

CN_Web_Server/

│

── index.html

── notes.txt

── sample.pdf

── image.jpg

Example index.html:

<html>

<head>

<title>Computer Networks Lab</title>

</head>

<body>

<h1>Welcome to CN Lab</h1>

This webpage is downloaded using TCP socket client.

</body>

</html>

Step 2: Start Python Web Server

Inside the folder:

python3 -m http.server 8000

Output:

Serving HTTP on 0.0.0.0 port 8000

Now your local URLs are:

http://localhost:8000/index.html

http://localhost:8000/sample.pdf

http://localhost:8000/image.jpg

http://localhost:8000/notes.txt

Step 3: C Client Requests URL

Example user input:

Enter URL:

http://localhost:8000/sample.pdf

Client extracts:

Host : localhost

Port : 8000

Path : /sample.pdf

Then sends:

GET /sample.pdf HTTP/1.1

Host: localhost

Connection: close

Step 4: Server Response

Python server sends:

HTTP/1.0 200 OK

Content-Type: application/pdf

Content-Length: xxxx

(binary PDF data)

The C client saves:

downloaded_sample.pdf

Advanced Program (optional)

Develop a secure HTTPS client application in **C programming** using **TCP sockets and OpenSSL APIs**. The client should establish a TCP connection with an HTTPS server (use example.com or google.com), perform SSL/TLS handshake, verify the server certificate, send

an HTTP GET request, receive the encrypted webpage response, and analyse the secure communication parameters

Objectives

The program should:

1. Establish a TCP connection with an HTTPS server (port 443).
2. Create an SSL/TLS secure channel over TCP.
3. Perform SSL handshake with the server.
4. Validate the server certificate.
5. Send HTTP GET requests securely.
6. Receive and decrypt webpage content.
7. Analyse security and performance parameters – TCP Connection Time, SSL Handshake time, Data Transfer Rate, Round Trip Time